# Reinforcement Learning | Markov Decision Processes

Selene · October 13, 2026

## 1 Markov Processes

**The Markov property** states that the future state depends only on the current state, not on the past history.

We can use a state-transition matrix $P$ to describe the transition probabilities $p(s_{t+1} = s' \mid s_t = s)$:

$$P = \begin{pmatrix} p(s_1 \mid s_1) & p(s_2 \mid s_1) & \cdots & p(s_N \mid s_1) \\ p(s_1 \mid s_2) & p(s_2 \mid s_2) & \cdots & p(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 \mid s_N) & p(s_2 \mid s_N) & \cdots & p(s_N \mid s_N) \end{pmatrix}.$$

## 2 Markov Reward Processes

A Markov reward process adds a reward function to a Markov chain. The reward function gives the expected one-step reward. For an episode with a finite number of steps, the **return** is the discounted sum of rewards. If the rewards after time $t$ are $r_{t+1}, r_{t+2}, r_{t+3}, \ldots$, then the return is

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T.$$

Here, $T$ is the final time step and $\gamma$ is the discount factor. When $0 < \gamma < 1$, rewards further in the future are discounted more heavily, expressing a preference for rewards received sooner. We define the **state-value function** as the expected return:

$$\begin{aligned} V^t(s) &= \mathbb{E}[G_t \mid s_t = s] \\ &= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T \mid s_t = s\right]. \end{aligned}$$

The superscript $t$ records the dependence on time in a finite-horizon episode.

### 2.1 The Bellman Equation

**The Bellman equation** decomposes the value of a state into two parts: the **expected reward for the next step**, plus the **discounted expected value of the next state**.

For a time-homogeneous process with an infinite discounted horizon, we write the stationary value function as $V(s)$:

$$V(s) = R(s) + \gamma \sum_{s'} p(s' \mid s) V(s').$$

Here, $R(s)$ is the expected one-step reward from state $s$, and $p(s' \mid s)$ is the probability of moving from $s$ to $s'$. The term $\gamma \sum_{s'} p(s' \mid s) V(s')$ represents the expected discounted return after the first step. For a finite horizon, the corresponding recursion uses $V^t$ on the left and $V^{t+1}$ on the right, with $V^T(s) = 0$.

We can write the Bellman equation in matrix form:

$$\begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix} = \begin{pmatrix} R(s_1) \\ R(s_2) \\ \vdots \\ R(s_N) \end{pmatrix} + \gamma \begin{pmatrix} p(s_1 \mid s_1) & p(s_2 \mid s_1) & \cdots & p(s_N \mid s_1) \\ p(s_1 \mid s_2) & p(s_2 \mid s_2) & \cdots & p(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 \mid s_N) & p(s_2 \mid s_N) & \cdots & p(s_N \mid s_N) \end{pmatrix} \begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix}.$$

Combining the Bellman equations for all states gives $V = R + \gamma P V$. Rearranging yields the **analytical solution**:

$$V = (I - \gamma P)^{-1} R.$$

For a finite state space and $0 \leq \gamma < 1$, the matrix $I - \gamma P$ is invertible.

### 2.2 Computing State Values

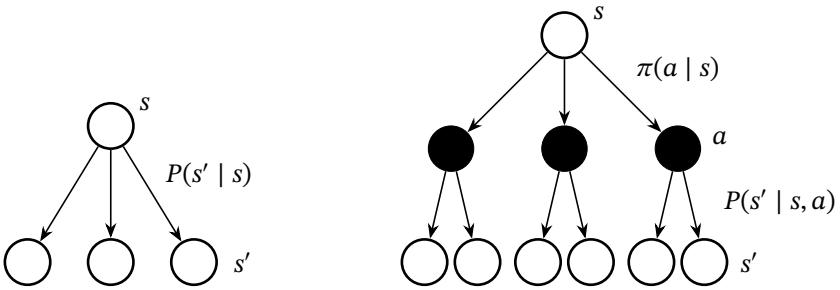
- **Monte Carlo:** Starting from state $s$, sample multiple complete trajectories, compute the return of each trajectory, and average those returns.

- **Dynamic programming:** When the rewards and transition probabilities are known, repeatedly apply the Bellman equation to update the values until they converge. Updating a state's value using the current estimates of its successor states is called **bootstrapping**.

# 3 Markov Decision Processes

## 3.1 MDPs and MRPs

In an MRP, the next state is sampled directly from the transition distribution over the state space $S$. In an MDP, an action comes first: the agent chooses an action $a$, and the next state is then sampled according to the transition distribution conditioned on both the state and the action. A policy $\pi(a \mid s)$ describes how the agent chooses actions in state $s$.




## 3.2 The Bellman Expectation Equations

For a fixed policy $\pi$, $V_\pi(s)$ is the expected return when we start in state $s$ and follow $\pi$. The action-value function $Q_\pi(s, a)$ is the expected return when we first take action $a$ in state $s$ and then follow $\pi$. They satisfy

$$V_\pi(s) = \sum_a \pi(a \mid s) Q_\pi(s, a),$$
$$Q_\pi(s, a) = R(s, a) + \gamma \sum_{s'} p(s' \mid s, a) V_\pi(s').$$

Here, $R(s, a)$ is the expected one-step reward for taking action $a$ in state $s$.

Substituting the first equation into the second gives a Bellman expectation equation involving only $Q_\pi$:

$$Q_\pi(s, a) = R(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') Q_\pi(s', a').$$

Similarly, we obtain a Bellman expectation equation involving only $V_\pi$:

$$V_\pi(s) = \sum_a \pi(a \mid s) \left[ R(s, a) + \gamma \sum_{s'} p(s' \mid s, a) V_\pi(s') \right].$$

# 4 Optimal Policies

## 4.1 The Optimal Value Function

The **optimal value function** is defined as $V^*(s) = \max_\pi V_\pi(s)$. An optimal policy $\pi^*$ attains this value for every state. Once we have the optimal action-value function $Q^*$, we can obtain an optimal policy by maximizing $Q^*$ in each state. Choose an action $a^*(s) \in \arg\max_{a \in A} Q^*(s, a)$, selecting one maximizer if there is a tie. The resulting deterministic policy is

$$\pi^*(a \mid s) = \begin{cases} 1, & a = a^*(s), \\ 0, & \text{otherwise.} \end{cases}$$

We now introduce two common methods for finding an optimal policy: **policy iteration** and **value iteration**. The following updates use the same finite-state, discounted setting as above.

## 4.2 Policy Iteration

Policy iteration consists of two important steps: **policy evaluation** and **policy improvement**.

Policy evaluation computes the state-value function of a policy. We then use those values to improve the policy, which is the policy improvement step. Policy iteration alternates between these steps until it converges to an optimal policy. Let us look at the two steps more closely.

**Policy evaluation:** Fix a policy $\pi$ and repeatedly update state values using the Bellman expectation equation:

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \left[ R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_k(s') \right].$$

**Policy improvement:** Using the values obtained from evaluation, choose an action with the highest expected return in each state:

$$\pi'(s) \in \arg\max_a \left[ R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_\pi(s') \right].$$

Here, $\pi'(s)$ denotes the action chosen by the improved deterministic policy. When actions tie, retaining the current action if it is a maximizer avoids unnecessary policy changes.

## 4.3 The Bellman Optimality Equations

The optimal state value and optimal action value satisfy

$$V^*(s) = \max_a Q^*(s,a),$$

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V^*(s').$$

Combining the two equations gives two common forms:

$$V^*(s) = \max_a \left[ R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V^*(s') \right],$$

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) \max_{a'} Q^*(s',a').$$

After computing the optimal values, choosing an action that maximizes $Q^*$ in each state gives an optimal policy:

$$\pi^*(s) \in \arg\max_a Q^*(s,a).$$

## 4.4 Value Iteration

Value iteration updates each state's value once per iteration and directly selects the action with the largest value during that update:

$$V_{k+1}(s) = \max_a \left[ R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_k(s') \right].$$

Starting from an arbitrary initial value function $V_0$, repeatedly apply this update to every state until the change between successive iterations is sufficiently small. At that point, $V$ is close to the optimal state-value function $V^*$. We then choose a policy greedily with respect to the resulting value estimates. With the exact optimal values, this gives

$$\pi^*(s) \in \arg\max_a \left[ R(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V^*(s') \right].$$

## 4.5 Comparing Policy Iteration and Value Iteration

- **Policy iteration:** Fix the current policy, repeatedly evaluate its value, and then improve it using the evaluation results. Alternate between evaluation and improvement until the policy no longer changes.
- **Value iteration:** Update each state's value once per iteration, choosing the action with the largest value under the current estimates. Repeat these updates until the values converge.

Policy iteration evaluates each policy more thoroughly before improving it, which can make its progress more stable. Value iteration is more like a greedy update at each step. Which method to choose depends on the setting.